COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	August 18, 2026	**DATE SUBMITTED:**	August 23, 2026
NAME:	Garbosa, Isaiah Benedict R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 6.1 Searching Techniques

1. PROCEDURE OUTPUT

ILO A: Create C++ code for searching data using different techniques

Screenshot

```
#include <iostream>
#include <cstdlib> //generating random int
#include <time.h> //seeding function
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main(){

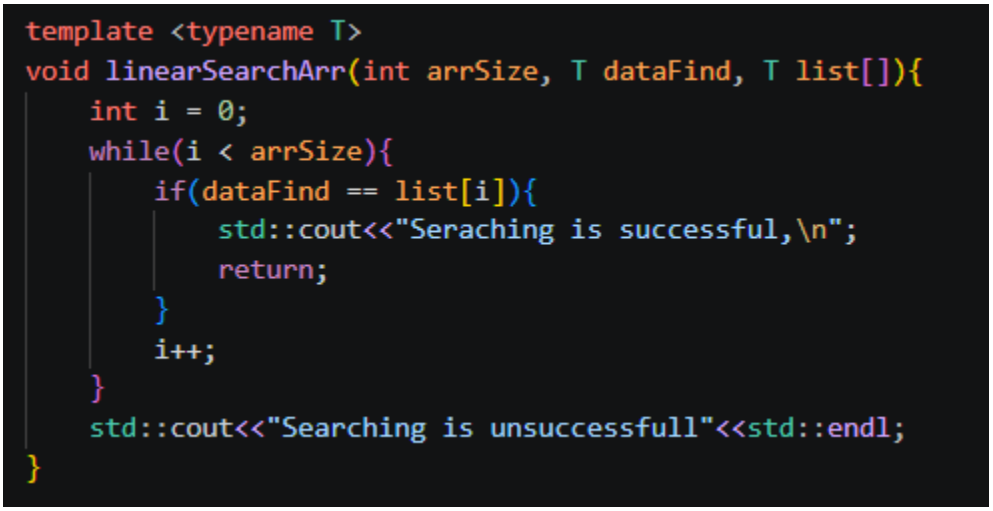
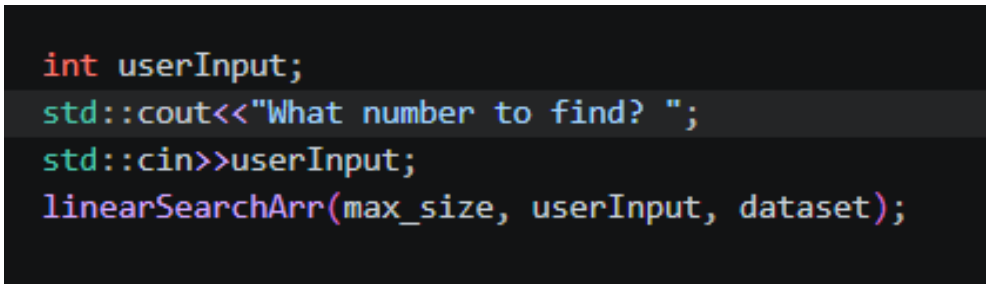
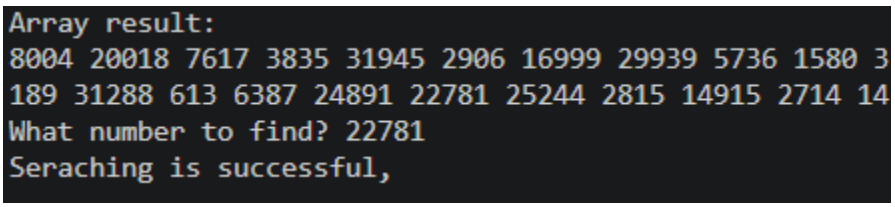
    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size; i++){
        dataset[i] = rand();
    }

    std::cout<<"Array result: "<<std::endl;
    for(int i = 0; i < max_size; i++){
        std::cout<<dataset[i]<<" ";
    }
    std::cout<<std::endl;
```

Observation

The code sets up the dataset to generate 50 random int values by using <cstdlib> and a for loop.

Table 6-1. Data Generated and Observations

Screenshot	 
Output	
Observation	The linear search goes through each of the element in an array and returns a statement saying search is successful upon getting a match with the value that the program is finding.

```
template <typename T>
void linearSearchArr(int arrSize, T dataFind, T list[]){
    int i = 0;
    while(i < arrSize){
        if(dataFind == list[i]){
            std::cout<<"Seraching is successful,\n";
            return;
        }
        i++;
    }
    std::cout<<"Searching is unsuccessfull"<<std::endl;
}
```

```
int userInput;
std::cout<<"What number to find? ";
std::cin>>userInput;
linearSearchArr(max_size, userInput, dataset);
```

```
Array result:
8004 20018 7617 3835 31945 2906 16999 29939 5736 1580 3
189 31288 613 6387 24891 22781 25244 2815 14915 2714 14
What number to find? 22781
Seraching is successful,
```

Table 6-2a. Linear Search for Arrays

Screenshot	

```cpp
#ifndef NODE_H
#define NODE_H

template <typename T>

class Node{
public:
    T data;
    Node *next;
};

template <typename T>
Node<T> *new_node(T newData){
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

template <typename T>
void linearSearchLL(Node<T>* head, T dataFind){
    while(head != nullptr){
        if(head->data == dataFind){
            std::cout<<"Searching is successful.\n";
            return;
        }
        head = head->next;
    }
    std::cout<<"Searching is Unsuccessful"<<std::endl;
};

#endif
```

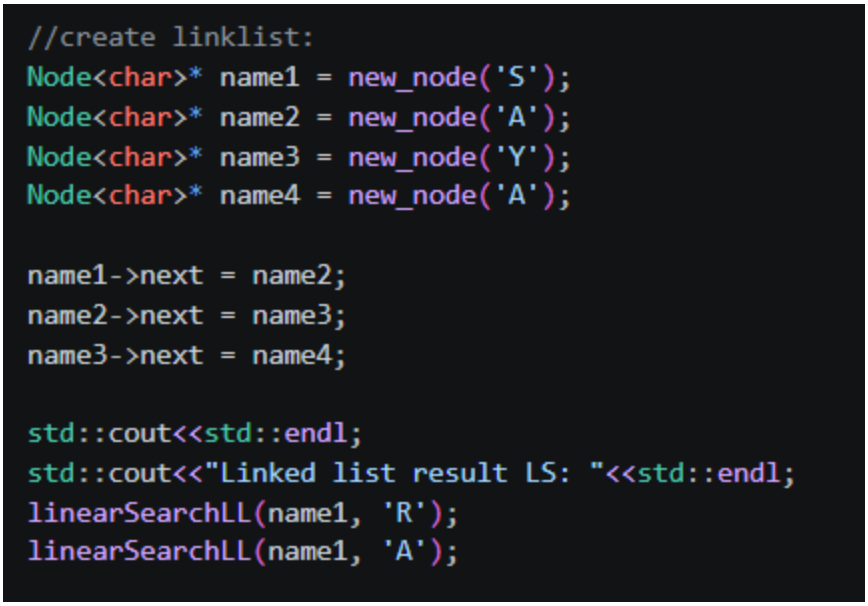
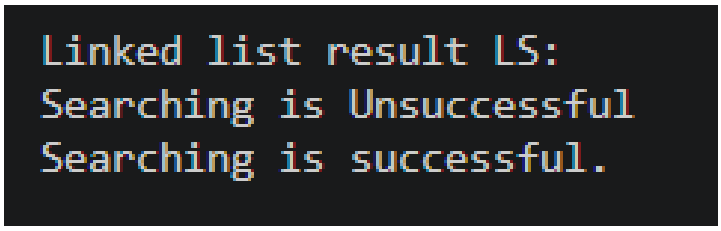
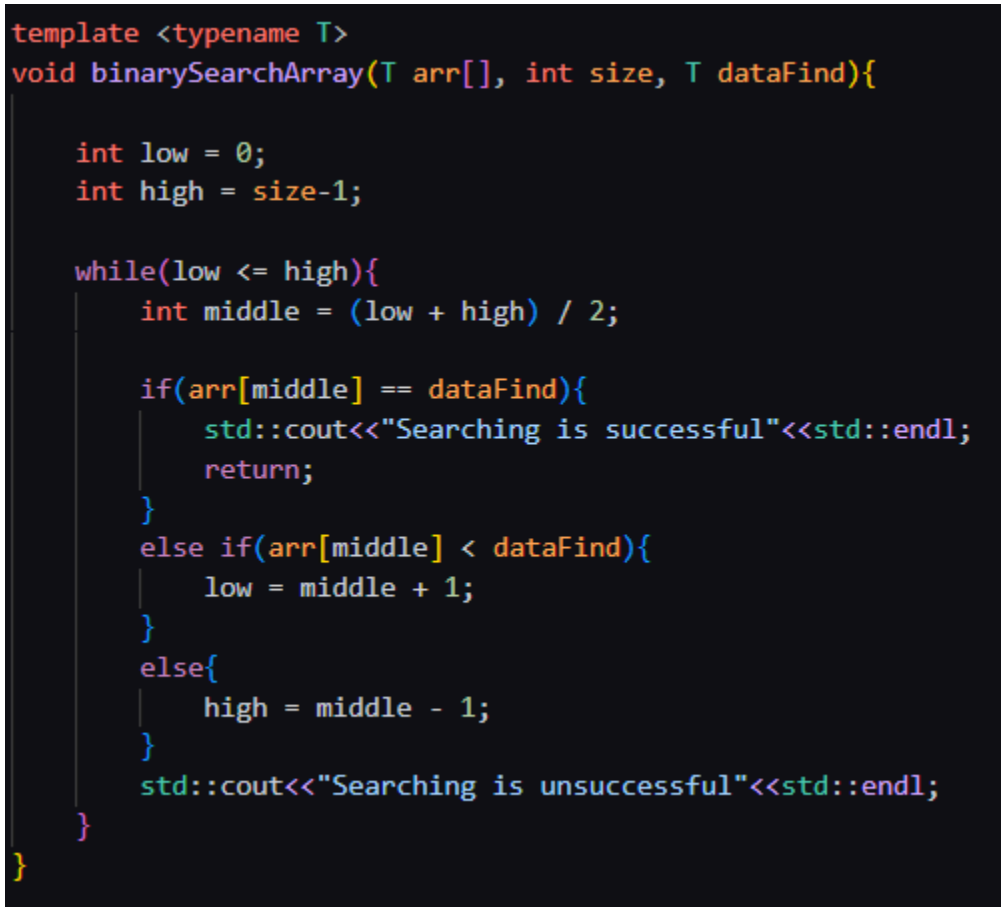
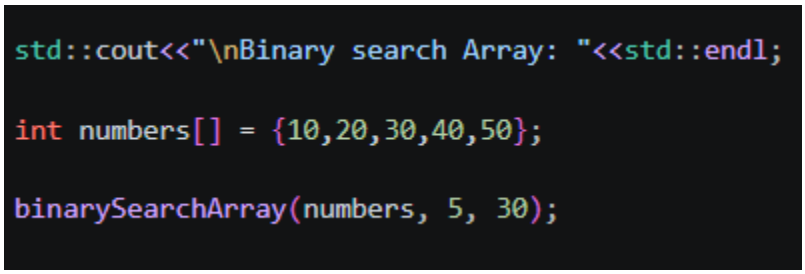
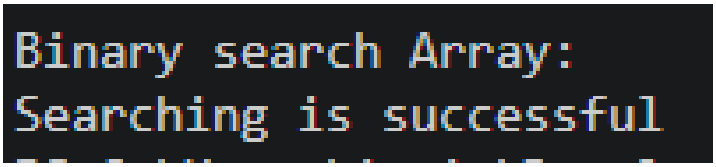
	
Output	
Observation	The linear search linked list uses nodes to traverse to the list. It uses a while loop, comparing each data of the current node to the data that the program is finding. If not, the current node is set to the next until it hits the null pointer.

Table 6-2b. Linear Search for Linked List

Screenshot	 
Output	
Observation	The binary search for an array gets the first and last index of the array. It gets the middle of the array by adding the low and high and dividing to 2. It will then compare the value we're finding to the middle. If the value is less than the middle, then the high is set to the middle minus 1. The array will be shortened and get a new middle. If the data find is greater than the middle, it will add to the low, eliminating the left side. Once the program finds a match, it will end.

```
template <typename T>
void binarySearchArray(T arr[], int size, T dataFind){

    int low = 0;
    int high = size-1;

    while(low <= high){
        int middle = (low + high) / 2;

        if(arr[middle] == dataFind){
            std::cout<<"Searching is successful"<<std::endl;
            return;
        }
        else if(arr[middle] < dataFind){
            low = middle + 1;
        }
        else{
            high = middle - 1;
        }
        std::cout<<"Searching is unsuccessful"<<std::endl;
    }
}
```

```
std::cout<<"\nBinary search Array: "<<std::endl;

int numbers[] = {10,20,30,40,50};

binarySearchArray(numbers, 5, 30);
```

```
Binary search Array:
Searching is successful
```

Table 6-3a. Binary Search for Arrays

Screenshot	

```
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    while(fast != last && fast->next != last){
        slow = slow->next;

        fast = fast->next->next;
    }
    return slow;
}
```

```
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){

    Node<T>* last = nullptr;

    while(start != last){
        Node<T>* middle = getMiddle(start, last);

        if (middle->data == dataFind){
            std::cout<<"Searching is successful"<<std::endl;
            return;
        }
        else if(middle->data < dataFind){
            start = middle->next;
        }
        else{
            last = middle;
        }
    }
    std::cout<<"Searching is unsuccessful"<<std::endl;
}
```

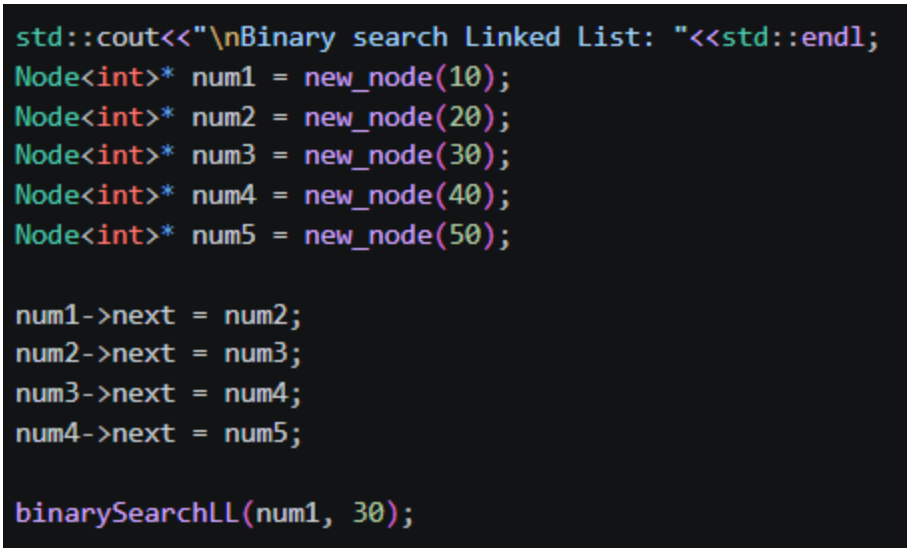
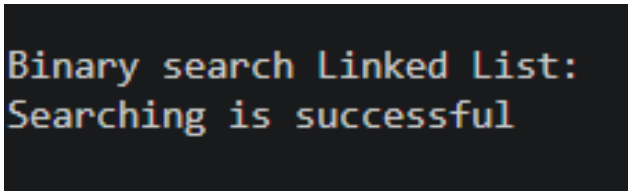
	
Output	
Observation	The binary search for the linked list first gets the middle.The searching is similar to the array. It compares the data that is finding to the middle and then if the data is less than the middle, it sets the middle to the last and if the data is higher than the middle, then the start is set to the middle next.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18

Code (array)

```
template <typename T>
int sequentialSearchArr(T arr[], int arrSize, T key){
    int comparisons = 0;

    for (int i = 0; i < arrSize; i++){
        comparisons++;

        if (arr[i] == key){
            return comparisons;
        }
    }
    return comparisons;
}
```

```
int arr[] = {15,18,2,19,18,0,8,14,19,14};
int key = 18;

int comparisons = sequentialSearchArr(arr, 2, key);

std::cout<<"Key: "<<key<<std::endl;
std::cout<<"Number of comparisons: "<<comparisons<<std::endl;
```

Output

```
Key: 18
Number of comparisons: 2
```

(for validation purposes)

```
Key: 8
Number of comparisons: 7
```

Code (linked list)

```
template <typename T>
int sequentialSearchLL(Node<T>* head, T key){
    int comparisons = 0;

    Node<T>* current = head;

    while (current != nullptr){
        comparisons++;

        if(current->data == key){
            return comparisons;
        }
        current = current->next;
    }
    return comparisons;
}
```

```cpp
//sequential search LL
Node<int>* head = nullptr;
Node<int>* tail = nullptr;

for (int i = 0; i < size; i++){
    Node<int>* newNode = new Node<int>;

    newNode->data = arr[i];
    newNode->next = nullptr;

    if (head == nullptr){
        head = newNode;
        tail = newNode;
    }
    else{
        tail->next = newNode;
        tail = newNode;
    }
};

int comparisonsLL = sequentialSearchLL(head, key);

std::cout<<std::endl;
std::cout<<"Key: "<<key<<std::endl;
std::cout<<"Number of comparisons: "<<comparisonsLL<<std::endl;
```

Output

```
Key: 18
Number of comparisons: 2
```

(for validation purposes)

```
Key: 8
Number of comparisons: 7
```

Answer
For both of the 2 sequential search methods, it takes **two** comparisons to get a match with the key. It does this by using a for or while loop to go to each value in the array or linked list, incrementing a comparison variable. When it gets a match, it displays the current comparison value.

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Code

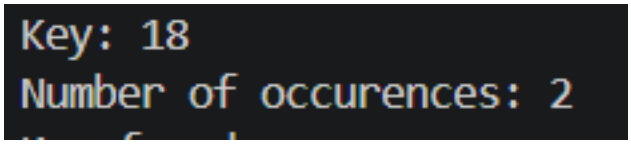
```
template <typename T>
int countOccurences(T arr[], int arrSize, T key){
    int count = 0;

    for(int i = 0; i < arrSize; i++){
        if(arr[i]==key){
            count++;
        }
    }
    return count;
}
```

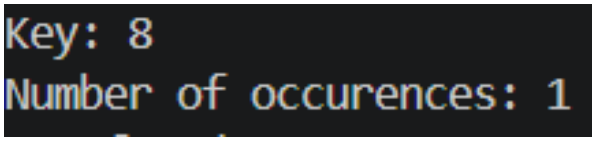
```
//counting occurences
int occurrences = countOccurences(arr, size, key);

std::cout<<std::endl;
std::cout<<"Key: "<<key<<std::endl;
std::cout<<"Number of comparisons: "<<occurrences<<std::endl;
```

Output



(for validation purposes):



Answer

The occurrence of the key in the array is 2 times based on the code. It is done by having a for loop go through each element in the array, incrementing to a count variable every time the current element is equal to the key.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Code

```cpp
template <typename T>
int binarySearch(T arr[], int arrSize, T key){

    int low = 0;
    int high = arrSize - 1;
    int comparisons = 0;

    while(low <= high){
        int middle = (low + high)/2;

        comparisons++;

        if(arr[middle] == key){
            std::cout<<"Key found"<<std::endl;
            return comparisons;
        }
        else if (key<arr[middle]){
            high = middle - 1;
        }
        else{
            low = middle + 1;
        }
    }
    return comparisons;
}
```

```cpp
//binary search
int arrB[] = {3,5,6,8,11,12,14,15,17,18};
int sizeB = 10;
int keyB = 8;

int comparisonsBS = binarySearch(arrB, sizeB, keyB);

std::cout<<std::endl;
std::cout<<"Number of comparisons: "<<comparisonsBS<<std::endl;
```

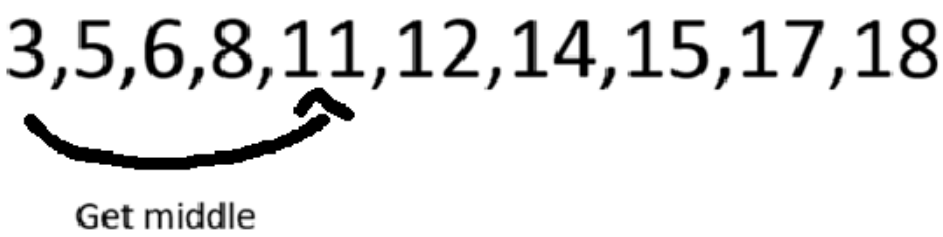
```cpp
//binary search
int arrB[] = {3,5,6,8,11,12,14,15,17,18};
int sizeB = 10;
int keyB = 8;

std::cout<<std::endl;
int comparisonsBS = binarySearch(arrB, sizeB, keyB);
std::cout<<"Number of comparisons: "<<comparisonsBS<<std::endl;
```

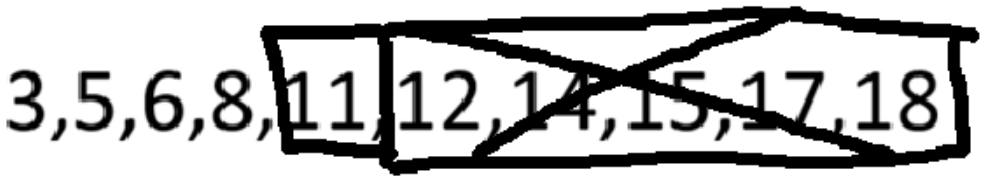
Output

```
Key 8 found at index 3
Number of comparisons: 4
```

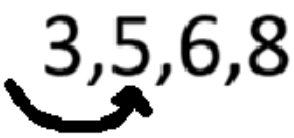
Answer



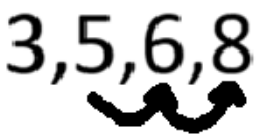
Get middle



Since key is less than 11, eliminate the right half and get new middle



New middle



Since key is greater than middle, we add until we reach the key

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Code

```
template <typename T>
int recursiveBinarySearch(T arr[], int low, int high, T key, int& comparisons){
    if (low > high){
        return -1;
    }

    int middle = (low + high) / 2;

    comparisons++;

    if (arr[middle] == key){
        return middle;
    }
    else if(key < arr[middle]){
        return recursiveBinarySearch(arr, low, middle - 1, key, comparisons);
    }
    else{
        return recursiveBinarySearch(arr, middle + 1, high, key, comparisons);
    }
}
```

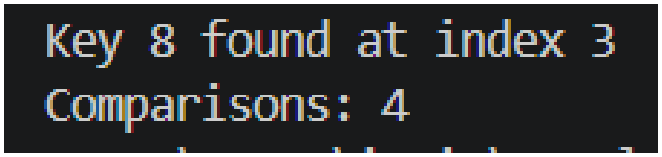
```
//recursive binary search
int comparisonsR = 0;

int result = recursiveBinarySearch(arrB,0,sizeB - 1, keyB, comparisonsR);

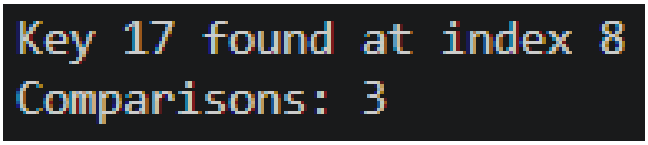
if(result != -1){
    std::cout<<"Key "<<keyB<<" found at index "<<result<<std::endl;
}
else{
    std::cout<<"Key not found."<<std::endl;
}

std::cout<<"Comparisons: "<<comparisonsR<<std::endl;
```

Output



(for validation purposes)



Answer

The recursive binary search functions the same as the ordinary binary search but instead calls itself. The key 8 is found at the third index with 4 comparisons.

Conclusion

The activity focused on the different methods of finding data in an array or linked list. I learned that there are two main methods of finding data, linear and binary. Linear search goes through each data in an unorganized dataset while binary search finds data in an organized dataset by getting the middle and comparing whether the data that we're finding is higher or lower. I believe I was able to learn a lot about the searching methods and would be able to apply them in the future. I'd like to practice more, try other methods and experiment with various different data sets.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.